

Unit-IV

Testing Strategies

What is Test Strategy?

Test strategy is a plan for defining an approach to the Software Testing Life Cycle (STLC). In addition, the test strategy provides the following details, which are required while writing the test document:

To put it another way, it's a document that explains the process of product evaluation. And the approaches can be developed using the following factors:

- Whether or not to automate
- From the standpoint of a resource

A Strategic Approach to Software Testing

1. Software testing is not a standalone phase but integrated throughout development.
2. It follows a **V-Model**—each development stage has a corresponding testing activity.
3. The goal is to **uncover errors** and ensure the system meets requirements.
4. Testing begins at the **component level** and moves outward to the entire system.
5. Each test level (unit, integration, validation, system) has specific objectives.
6. A **formal test strategy** helps manage time, cost, and quality.
7. Early testing helps reduce risk and cost of fixing defects later.
8. It emphasizes **test planning**, **test case design**, and **test data preparation**.
9. Testing is both a **verification** (building the product right) and **validation** (building the right product) activity.
10. Strategic testing ensures a high-quality, reliable, and maintainable software product.

Test Strategies for Conventional Software

1. Testing is applied in a systematic and phased manner.
2. Unit Testing validates individual components or modules.
3. Integration Testing checks data flow and interaction between modules.

4. Validation Testing ensures the software meets customer requirements.
 5. System Testing evaluates the entire software as a complete system.
 6. Strategies emphasize early detection of errors to reduce rework cost.
 7. Each phase builds on the previous one in a layered fashion.
 8. Test plans and test cases are designed based on software requirements.
 9. Ensures both functional and non-functional aspects are verified.
 10. Supports high software quality, reliability, and user satisfaction.
-

Black-Box Testing

1. **Focuses on external behavior of the software.**
 2. **Internal code or logic is not considered.**
 3. **Test cases are based on input/output conditions.**
 4. **Uses techniques like Equivalence Partitioning and Boundary Value Analysis.**
 5. **Helps detect functional errors, interface issues, and data handling bugs.**
 6. **Suitable for validation and system-level testing.**
 7. **Ideal when source code is unavailable.**
 8. **Tests what the software does, not how it does it.**
 9. **Ensures software meets user expectations.**
 10. **Easy to automate for repeated runs.**
-

White-Box Testing

1. **Focuses on the internal logic and structure of code.**
2. **Requires access to the source code.**
3. **Test cases are designed to achieve high code coverage.**
4. **Includes path testing, loop testing, and condition coverage.**
5. **Helps find logical errors and boundary conditions.**
6. **Ensures all code paths are tested.**
7. **Detects hidden bugs in control flow and logic.**
8. **Supports debugging and optimization.**

9. Generally performed by developers.

10. Answers the question: “Is the code doing it right?”

Validation Testing

1. Ensures that the software meets user requirements.

2. Confirms that the right product has been built.

3. Follows successful integration and system testing.

4. Includes alpha and beta testing with real users.

5. Detects missing or misunderstood features.

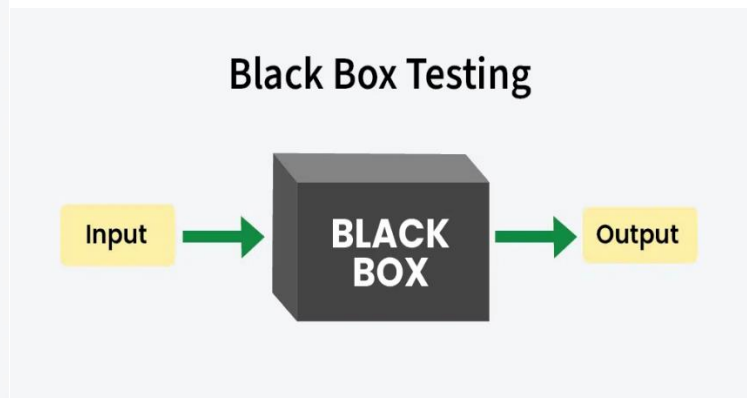
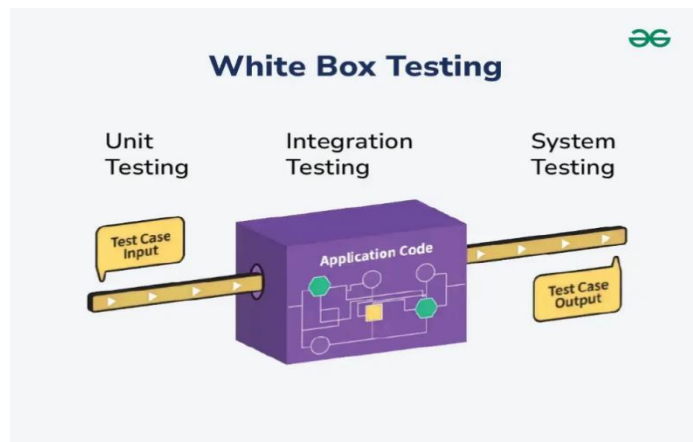
6. Validates both functional and non-functional requirements.

7. Often performed by a quality assurance team.

8. Helps gain customer confidence before release.

9. Answers the question: “Are we building the right product?”

10. A key part of acceptance testing.





Metrics for Software Quality

Metrics help assess software attributes like reliability, maintainability, and performance.

Examples: Defect density, Mean Time to Failure (MTTF), Cyclomatic complexity.

Helps in monitoring project health and product quality.

Used in process improvement and decision-making.

✂ Diagram:

□ Radar chart showing different quality metrics.

1. Black box testing – Tests the functionality of the software without looking at the internal code structure.
2. White box testing – Tests the internal code structure and logic of the software.
3. Unit testing – Tests individual units or components of the software to ensure they are functioning as intended.
4. Integration testing – Tests the integration of different components of the software to ensure they work together as a system.
5. Functional testing – Tests the functional requirements of the software to ensure they are met.
6. System testing – Tests the complete software system to ensure it meets the specified requirements.
7. Acceptance testing – Tests the software to ensure it meets the customer's or end-user's expectations.
8. Regression testing – Tests the software after changes or modifications have been made to ensure the changes have not introduced new defects.
9. Performance testing – Tests the software to determine its performance characteristics such as speed, scalability, and stability.
10. Security testing – Tests the software to identify vulnerabilities and ensure it meets security requirements.



System Testing

Tests the entire integrated system to verify it meets specifications.

Covers functional and non-functional aspects (like security, performance).

Includes stress testing, load testing, regression testing.

Simulates real-world usage scenarios.

Final step before product release.

✂ Diagram:

□ System architecture with arrows showing integrated component testing.

The Art of Debugging

Debugging is the process of finding and fixing defects.

It involves identifying the cause of errors using tools or manual methods.

Common techniques: backtracking, cause elimination, binary partitioning.

Debuggers and log analyzers help track down issues.

It's both logical and creative—hence “art”.

✂ Diagram:

□ Flowchart: Error → Trace → Locate Bug → Fix → Retest.

Software Measurement

Measurement is essential for managing software processes and products.

Includes size (LOC, Function Points), effort, cost, and complexity.

Helps in estimation, tracking, and quality assurance.

Quantifies various software aspects for better control.

✎ Diagram:

Pyramid of Software Measurement: Size, Effort, Quality.